ISO/IEC JTC1 SC22 WG21 P2314R3
Author: Jens Maurer
Target audience: CWG, LWG
2021-09-14

P2314R3: Character sets and encodings

Introduction

This paper implements the following changes:

- Switch C++ to a modified "model C" approach for *universal-character-name*s as described in the C99 Rationale v5.10, section 5.2.1.
- Introduce the term "literal encoding". For purposes of the C++ specification, the actual set of characters is not relevant, but the sequence of code units (i.e. the encoding) specified by a given character or string literal are. The terms "execution (wide) character set" are retained to describe the locale-dependent runtime character set used by functions such as `isalpha`.
- (Not a wording change) Do not attempt to treat all string literals the same; their treatment depends on (phase 7) context.

This paper resolves the following core issues:

- 578. Phase 1 replacement of characters with universal-character-names
- 1332. Handling of invalid universal-character-names
- 1335. Stringizing, extended characters, and universal-character-names
- 1403. Universal-character-names in comments
- 2455. Concatenation of string literals vs translation phases 5 and 6

Change history

Changes since R0

- missed edits in the normative wording
- add comparison with P2297R0
- retained "execution (wide) character set" for locale-dependent runtime encoding, but moved the definition to the library wording

Changes since R1

- missed edits in the normative wording
- Clarify that an *r-char-sequence* never contains a *universal-character-name*.
- Remove words that claim string literal objects are initialized in translation phase 5.
- Add SG16 and EWG poll results.
- Fix typo in [lex.ccon] table 10.

Changes since R2

- Feedback from CWG teleconference on 2021-09-14

Terminology changes

The following terms are defined by this paper:

- translation character set: the abstract character set used during translation; can represent the character equivalent of all valid *universal-character-name*s
- basic character set: minimum character set needed to express C++ program source
- basic literal character set: minimum set of characters expressible by literals
- ordinary / wide literal encoding: compile-time encoding used for initializing string literal objects

The term "basic / extended source character set" is removed.

Behavior changes

The core behavior change is that *universal-character-name*s are no longer formed in translation phase 1. Instead, all Unicode input characters are retained throughout the translation.

This changes the specified behavior of the stringizing preprocessor operator [cpp.stringize] as follows

C++20	this paper
`#define S(x) # x` `const char * s1 = S(Köppe);  // "K\\u00f6ppe"`	`#define S(x) # x` `const char * s1 = S(Köppe);  // "Köppe"`

`const char * s2 = S(K\u00f6ppe);  // "K\\u00f6ppe"`	`const char * s2 = S(K\u00f6ppe);  // "Köppe"`

However, it turns out that all major implementations already implement what this paper specifies, i.e. no implementation provides an escaped UCN.

Not all *string-literal*s are the same

In C++, string literals can appear in the following contexts:

Context	Destination
asm-declaration	build environment
`#include "fn"` or `#include <fn>`	file name
language linkage	translation
`operator ""` [over.literal]	translation
`#line` directive	diagnostic
argument for [[nodiscard]] and [[deprecated]]	diagnostic
`#error, static_assert`	diagnostic
__FILE__, __func__	literal encoding
`std::typeinfo::name()`	literal encoding
character-literal or *string-literal* appearing elsewhere	literal encoding
user-defined-literal	literal encoding

The destinations have the following meaning:

- build environment: A string likely passed as text (program input) to another component in the build environment.
- file name: A file name suitable for the build environment.
- translation: No use outside the compiler.
- diagnostic: Diagnostic text; kept in the translation character set until needed for output; then treated the same as e.g. *identifier*s appearing in diagnostic messages
- literal encoding: The implementation-defined encoding for the runtime environment, used for *string-literal*s that appear in usual program text.

The existing text in 5.13.5 [lex.string] already specifies that the initialization of a string literal object (as needed when using a *string-literal* as a primary expression) is the point where the *string-literal* is encoded. In other contexts, no such encoding happens. Words to the contrary appearing for translation phase 5 [lex.phases] have been excised.

Comparison with P2297R0

This paper P2314 refomulates the core language rules around lexing of non-basic characters, while keeping the actual semantic changes to a minimum. This makes it more likely that the paper can either directly proceed to CWG or be reviewed by EWG with minimal effort.

The paper P2297R0 "Wording improvements for encodings and character sets" by Corentin Jabot has overlap with this paper. The main differences are:

- terminology: This paper uses "translation character set"; P2297R0 uses "Unicode" to describe the C++ compile-time character set. Note that ISO 10646 does not seem to define "Unicode" as a stand-alone term, and the term seems unclear regarding the inclusion of non-assigned code points.
- alert and backspace for literals: This paper retains the status quo, after reformulation to refer to Unicode code points; P2297R0 removes the requirement to represent these. In my view, since *simple-escape-sequence*s are expressly specified to represent alert and backspace, those characters should be required to be representable in the literal encoding. The presence or absence of such a specified requirement is believed not to have an impact on currently existing implementations.

Poll results

SG16

Poll: Introduce the concept of a 'translation character set' which synthesizes characters for unassigned UCS scalar values.

```
SF F N A SA
 2 4 1 0  1
```

Present: 9
Consensus: In favour
P2314 author's position: F
Strongly against: The abstraction is unnecessary and the definition of 'translation character set' is incorrectly using terms defined by Unicode and UCS.

Poll: Forward D2314R2 as presented on 2021-03-24 to EWG for inclusion in C++23.

```
SF F N A SA
 3 5 0 0  0
```

Present: 9
Consensus: Strongly in favour
P2314 author's position: In favour

EWG

Send P2314 to Electronic Polling, with the intent of going to Core for C++23.

```
SF      F       N       A       SA
5       6       0       0       0
```

Wording changes

Change in 3.35 [defns.multibyte]:

> **multibyte character**
>
> sequence of one or more bytes representing ~~a member of the extended character set of either the source or the execution environment~~ **the code unit sequence for an encoded character of the execution character set**
>
> ~~[Note 1 to entry: The extended character set is a superset of the basic character set (5.3). — end note]~~

Change in 5.2 [lex.phases] paragraph 1:

> 1. Physical source file characters are mapped, in an implementation-defined manner, to the ~~basic source~~ **translation** character set (introducing new-line characters for end-of-line indicators) ~~if necessary~~. The set of physical source file characters accepted is implementation-defined. ~~Any source file character not in the basic source character set (5.3 [lex.charset]) is replaced by the universal-character-name that designates that character. An implementation may use any internal encoding, so long as an actual extended character encountered in the source file, and the same extended character expressed in the source file as a universal-character-name (e.g., using the \uXXXX notation), are handled equivalently except where this replacement is reverted (5.4 [lex.pptoken]) in a raw string literal.~~
>
> ...
>
> 3. The source file is decomposed into preprocessing tokens (5.4 [lex.pptoken]) and sequences of white-space characters (including comments). A source file shall not end in a partial preprocessing token or in a partial comment. [Footnote: ...] Each comment is replaced by one space character. New-line characters are retained. Whether each nonempty sequence of white-space characters other than new-line is retained or replaced by one space character is unspecified. **Each *universal-character-name* outside of a *header-name* or a character or string literal is replaced by the designated element of the translation character set ([lex.charset]).** The process of dividing a source file's characters into preprocessing tokens is context-dependent. [Example: See the handling of < within a #include preprocessing directive. — end example]
>
> 4. Preprocessing directives are executed, macro invocations are expanded, and _Pragma unary operator expressions are executed. ~~If a character sequence that matches the syntax of a *universal-character-name* is produced by token concatenation (15.6.3 [lex.concat]), the behavior is undefined.~~ A #include preprocessing directive causes the named header or source file to be processed from phase 1 through phase 4, recursively. All preprocessing directives are then deleted.
>
> 5. **For a sequence of two or more adjacent *string-literal* tokens, a common *encoding-prefix* is determined as specified in 5.13.5 [lex.string]. Each such *string-literal* token is then considered to have that common *encoding-prefix*.** ~~Each *basic-c-char*, *basic-s-char*, and *r-char* in a *character-literal* or a *string-literal*, as well as each *escape-sequence* and *universal-character-name* in a *character-literal* or a non-raw string literal, is encoded in the literal's associated character encoding as specified in 5.13.3 [lex.ccon] and 5.13.5 [lex.string].~~
>
> 6. Adjacent ~~string literal~~ ***string-literal*** tokens are concatenated ~~and a null character is appended to the result as specified in 5.13.5~~ **(5.13.5 [lex.string])**.

Replace all of 5.3 [lex.charset] (paragraphs 1-3):

> 1 The *translation character set* consists of the following elements:
>
> - each character named by ISO/IEC 10646, as identified by its unique UCS scalar value, and
> - a distinct character for each UCS scalar value where no named character is assigned.
>
> [Note: ISO/IEC 10646 code points are integers in the range [0, 10FFFF] (hexadecimal). A surrogate code point is a value in the range [D800, DFFF] (hexadecimal). A UCS scalar value is any code point that is not a surrogate code point. -- end note]
>
> 2 The *basic character set* is a subset of the translation character set, consisting of 96 characters as specified in table X. [Note: Unicode short names are given only as a means to identifying the character; the numerical value has no other meaning in this context. -- end note]
>
> | | | |
> |---|---|---|
> | U+0009 | CHARACTER TABULATION | |
> | U+000B | LINE TABULATION | |
> | U+000C | FORM FEED (FF) | |

U+0020	SPACE	
U+000A	LINE FEED (LF)	*new-line*
U+0021	EXCLAMATION MARK	!
U+0022	QUOTATION MARK	"
U+0023	NUMBER SIGN	#
U+0025	PERCENT SIGN	%
U+0026	AMPERSAND	&
U+0027	APOSTROPHE	'
U+0028	LEFT PARENTHESIS	(
U+0029	RIGHT PARENTHESIS	)
U+002A	ASTERISK	*
U+002B	PLUS SIGN	+
U+002C	COMMA	,
U+002D	HYPHEN-MINUS	-
U+002E	FULL STOP	.
U+002F	SOLIDUS	/
U+0030 .. U+0039	DIGIT ZERO .. NINE	0 1 2 3 4 5 6 7 8 9
U+003A	COLON	:
U+003B	SEMICOLON	;
U+003C	LESS-THAN SIGN	<
U+003D	EQUALS SIGN	=
U+003E	GREATER-THAN SIGN	>
U+003F	QUESTION MARK	?
U+0041 .. U+005A	LATIN CAPITAL LETTER A .. Z	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
U+005B	LEFT SQUARE BRACKET	[
U+005C	REVERSE SOLIDUS	\
U+005D	RIGHT SQUARE BRACKET	]
U+005E	CIRCUMFLEX ACCENT	^
U+005F	LOW LINE	_
U+0061 .. U+007A	LATIN SMALL LETTER A .. Z	a b c d e f g h i j k l m n o p q r s t u v w x y z
U+007B	LEFT CURLY BRACKET	{
U+007C	VERTICAL LINE	|
U+007D	RIGHT CURLY BRACKET	}
U+007E	TILDE	~

The *universal-character-name* construct provides a way to name other characters.

```
hex-quad :
    hexadecimal-digit hexadecimal-digit hexadecimal-digit hexadecimal-digit

universal-character-name :
    \u hex-quad
    \U hex-quad hex-quad
```

A *universal-character-name* designates the character in ~~ISO/IEC 10646 (if any)~~ **the translation character set** whose ~~Unicode code point~~ **UCS scalar value** is the hexadecimal number represented by the sequence of *hexadecimal-digit*s in the *universal-character-name*. The program is ill-formed if that number is not a ~~Unicode code point or if it is a surrogate code point~~ **UCS scalar value**. ~~Noncharacter code points and reserved code points are considered to designate separate characters distinct from any ISO/IEC 10646 character.~~ If a *universal-character-name* outside the *c-char-sequence*, *s-char-sequence*, or *r-char-sequence* of a *character-literal* or *string-literal* (in either case, including within a user-defined-literal) corresponds to a control character or to a character in the basic ~~source~~ character set, the program is ill-formed. [~~Footnote:~~ **Note:** A sequence of characters resembling a *universal-character-name* in an *r-char-sequence* (5.13.5) does not form

a *universal-character-name*.] ~~[Note: ISO/IEC 10646 code points are integers in the range [0, 10FFFF] (hexadecimal). A surrogate code point is a value in the range [D800, DFFF] (hexadecimal). A control character is a character whose code point is in either of the ranges [0, 1F] or [7F, 9F] (hexadecimal). — end note]~~

The *basic literal character set* consists of all characters of the basic character set, plus the control characters specified in table Y.

U+0000	NULL
U+0007	BELL
U+0008	BACKSPACE
U+000D	CARRIAGE RETURN (CR)

A *code unit* is an integer value of character type (6.8.1 [basic.fundamental]). Characters in a *character-literal* other than a multicharacter or non-encodable character literal or in a *string-literal* are encoded as a sequence of one or more code units, as determined by the *encoding-prefix* ([lex.ccon], [lex.string]); this is termed the respective *literal encoding*. The *ordinary literal encoding* is the encoding applied to an ordinary character or string literal. The *wide literal encoding* is the encoding applied to a wide character or string literal.

A literal encoding encodes each element of the basic literal character set as a single code unit with non-negative value, distinct from the code unit for any other such element. [Note: A character not in the basic literal character set can be encoded with more than one code unit; the value of such a code unit can be the same as that of a code unit for an element of the basic literal character set. -- end note]. The U+0000 NULL character is encoded as the value 0. No other element of the translation character set is encoded with a code unit of value 0. The code unit value of each decimal digit character after the digit 0 (U+0030) shall be one greater than the value of the previous. The ordinary and wide literal encodings are otherwise implementation-defined. For a UTF-8, UTF-16, or UTF-32 literal, the UCS scalar value corresponding to each character of the translation character set is encoded as specified in ISO/IEC 10646 for the respective UCS encoding form.

~~The basic execution character set and the basic execution wide-character set shall each contain all the members of the basic source character set, plus control characters representing alert, backspace, and carriage return, plus a null character (respectively, null wide character), whose value is 0. For each basic execution character set, the values of the members shall be non-negative and distinct from one another. In both the source and execution basic character sets, the value of each character after 0 in the above list of decimal digits shall be one greater than the value of the previous. The *execution character set* and the *execution wide-character set* are implementation-defined supersets of the basic execution character set and the basic execution wide-character set, respectively. The values of the members of the execution character sets and the sets of additional members are locale-specific.~~

Change in 5.4 [lex.pptoken] paragraph 2:

A preprocessing token is the minimal lexical element of the language in translation phases 3 through 6. **In this document, glyphs are used to identify elements of the basic character set ([lex.charset]).** The categories of preprocessing token are: header names, placeholder tokens produced by preprocessing import and module directives (*import-keyword*, *module-keyword*, and *export-keyword*), identifiers, preprocessing numbers, character literals (including user-defined character literals), string literals (including user-defined string literals), preprocessing operators and punctuators, and single non-whitespace characters that do not lexically match the other preprocessing token categories. If a ~~' or a "~~**U+0027 APOSTROPHE or a U+0022 QUOTATION MARK** character matches the last category, the behavior is undefined. Preprocessing tokens can be separated by whitespace; this consists of comments (5.7), or whitespace characters (~~space, horizontal tab~~**U+0020 SPACE, U+0009 CHARACTER TABULATION**, new-line, ~~vertical tab, and form-feed~~**U+000B LINE TABULATION, and U+000C FORM FEED**), or both. ...

Change in 5.4 [lex.pptoken] paragraph 3 bullet 1:

- If the next character begins a sequence of characters that could be the prefix and initial double quote of a raw string literal, such as R", the next preprocessing token shall be a raw string literal. Between the initial and final double quote characters of the raw string, any transformations performed in ~~phases~~ **phase** ~~1 and~~ 2 (~~universal-character-names and~~ line splicing) are reverted; this reversion shall apply before any d-char, r-char, or delimiting parenthesis is identified.

Change in 5.8 [lex.header] paragraph 1:

```
h-char:
    any member of the ~~source~~ translation character set except new-line and ~~>~~ U+003E GREATER-THAN SIGN
...
q-char:
    any member of the ~~source~~ translation character set except new-line and ~~"~~ U+0022 QUOTATION MARK
```

Change in 5.10 [lex.name]:

```
identifier-nondigit:
    nondigit
    any member of the translation character set from Table 2
    universal-character-name
```

Change in 5.13.3 [lex.ccon] before paragraph 1:

```
basic-c-char:
    any member of the ~~basic source~~ translation character set
    except the ~~single quote ', backslash \~~ U+0027 APOSTROPHE, U+005C REVERSE SOLIDUS, or new-line character
...
conditional-escape-sequence-char:
```

```
    any member of the basic source character set that is not an octal-digit, a simple-escape-sequence-char, or
    the characters u, U, or x
```

Change in 5.13.3 [lex.ccon] paragraph 2:

[Note 1 : The associated character encoding for ordinary and wide character literals determines encodability, but does not determine the value of non-encodable ordinary or wide character literals or ordinary or wide multicharacter literals. The examples in Table 9 for non-encodable ordinary and wide character literals assume that the specified character lacks representation in the ~~execution character set~~ **ordinary literal encoding** or ~~execution wide-character set~~ **wide literal encoding**, respectively, or that encoding it would require more than one code unit. — end note]

Change in 5.13.3 [lex.ccon] table tab:lex.ccon.literal:

Encoding prefix	...	Associated character encoding
none	...	~~encoding of the execution character set~~ **ordinary literal encoding**
L	...	~~encoding of the execution wide-character set~~ **wide literal encoding**

Replace 5.13.3 [lex.ccon] table tab:lex.ccon.esc:

The character specified by a *simple-escape-sequence* is specified in Table 10.

character		***simple-escape-sequence***
U+000A	LINE FEED (LF)	`\n`
U+0009	CHARACTER TABULATION	`\t`
U+000B	LINE TABULATION	`\v`
U+0008	BACKSPACE	`\b`
U+000D	CARRIAGE RETURN (CR)	`\r`
U+000C	FORM FEED (FF)	`\f`
U+0007	BELL	`\a`
U+005C	REVERSE SOLIDUS	`\\`
U+003F	QUESTION MARK	`\?`
U+0027	APOSTROPHE	`\'`
U+0022	QUOTATION MARK	`\"`

Change in 5.13.5 [lex.string] before paragraph 1:

```
basic-s-char:
    any member of the ~~basic source~~ translation character set
    except the ~~double quote ", backslash \~~U+0022 QUOTATION MARK, U+005C REVERSE SOLIDUS, or new-line character
...
r-char:
    any member of the ~~source~~ translation character set, except a ~~right parenthesis )~~ U+0029 RIGHT PARENTHESIS followed by
    the initial d-char-sequence (which may be empty) followed by a ~~double quote "~~ U+0022 QUOTATION MARK.
...
d-char:
    any member of the basic ~~source~~ character set except:
    ~~space, the left parenthesis (, the right parenthesis ), the backslash \, and the control characters~~
    ~~representing horizontal tab, vertical tab, form feed~~
    U+0020 SPACE, U+0028 LEFT PARENTHESIS, U+0029 RIGHT PARENTHESIS, U+005C REVERSE SOLIDUS,
    U+0009 CHARACTER TABULATION, U+000B LINE TABULATION, U+000C FORM FEED (FF), and new-line

and the control characters
representing horizontal tab, vertical tab, form feed, and newline.
```

Change in 5.13.5 [lex.string] table tab:lex.string.literal:

Encoding prefix	...	Associated character encoding
none	...	~~encoding of the execution character set~~ **ordinary literal encoding**
L	...	~~encoding of the execution widecharacter set~~ **wide literal encoding**

Change in 5.13.5 [lex.string] paragraphs 7 and 8:

- 7 - ~~In translation phase 6 (5.2 [lex.phases]), adjacent *string-literals* are concatenated.~~ **The common *encoding-prefix* for a sequence of adjacent *string-literal*s is determined pairwise as follows:** If ~~both~~ **two** *string-literal*s have the same *encoding-prefix*, the ~~resulting concatenated *string-literal* has~~ **common *encoding-prefix* is** that *encoding-prefix*. If one *string-literal* has no *encoding-prefix*, ~~it is treated as~~

~~a *string-literal* of the same *encoding-prefix* as~~ **the common *encoding-prefix* is that of** the other ~~operand~~ ***string-literal***. If a UTF-8 string literal token is adjacent to a wide string literal token, the program is ill-formed. Any other ~~concatenations~~ **combinations** are conditionally-supported with implementation-defined behavior. [Note: ~~This concatenation is an interpretation, not a conversion. Because the interpretation happens in translation phase 6 (after each character from a string literal has been translated into a value from the appropriate character set), a~~ **A** *string-literal*'s ~~initial~~ rawness has no effect on the ~~interpretation or well-formedness of the concatenation~~ **determination of the common *encoding-prefix***. -- end note]

~~Table 13 has some examples of valid concatenations.~~

- 8 - **In translation phase 6 (5.2 [lex.phases]), adjacent *string-literal*s are concatenated. The lexical structure of the contents of the individual *string-literal*s is retained.** ~~Characters in concatenated strings are kept distinct.~~ [Example:

```
"\xA" "B"
```

~~contains the two characters~~ **represents the code unit** '\xA' and **the character** 'B' after concatenation (and not the single ~~hexadecimal character~~ **code unit** '\xAB'). **Similarly,**

```
R"(\u00)" "41"
```

represents six characters, starting with a backslash and ending with the digit `1` (and not the single character "A" specified by a *universal-character-name*).

Table 13 has some examples of valid concatenations. — end example]

~~In translation phase 6 (5.2), after adjacent string literals are concatenated, a null character is appended to the result.~~

Change in 5.13.5 [lex.string] paragraph 10 and de-bulletize:

String literal objects are initialized with the sequence of code unit values corresponding to the *string-literal*'s sequence of *s-char*s (for a non-raw string literal) and *r-char*s (for a raw string literal)**, plus a terminating U+0000 NULL character,** in order as follows:

- The sequence of characters denoted by each contiguous sequence of *basic-s-char*s, *r-char*s, *simple-escape-sequence*s (5.13.3), and *universal-character-name*s (5.3) is encoded to a code unit sequence using the *string-literal*'s associated character encoding. If a character lacks representation in the associated character encoding, ~~then: If the *string-literal*'s *encoding-prefix* is absent or `L`,~~ then the *string-literal* is conditionally-supported and an implementation-defined code unit sequence is encoded. **[Note: No character lacks representation in any of the UCS encoding forms. -- end note]** ~~Otherwise, the *string-literal* is ill-formed.~~

  When encoding a stateful character encoding, ...

- ...

Change in 5.13.8 [lex.ext] paragraph 3:

[Note: The sequence c_1 c_2 ...c_k can only contain characters from the basic ~~source~~ character set. — end note]

Change in 5.13.8 [lex.ext] paragraph 4:

[Note: The sequence c_1 c_2 ...c_k can only contain characters from the basic ~~source~~ character set. — end note]

Change in 6.7.1 [intro.memory] paragraph 1:

The fundamental storage unit in the memory model is the *byte*. A byte is at least large enough to contain ~~any member~~ **the ordinary literal encoding of any element** of the basic ~~execution~~ **literal** character set (5.3) and the eight-bit code units of the Unicode UTF-8 encoding form and is composed of a contiguous sequence of bits, [Footnote: ...] the number of which is implementation-defined.

Change in 6.8.2 [basic.fundamental] paragraph 7:

Type `char` is a distinct type that has an implementation-defined choice of "signed char" or "unsigned char" as its underlying type. ~~The values of type `char` can represent distinct codes for all members of the implementation's basic character set.~~ ...

Editing note: The strike-out above is already stated in the definition of "byte", above. If desired, we can add a note that a char takes exactly one byte.

Change in 6.8.2 [basic.fundamental] paragraph 8:

Type `wchar_t` is a distinct type that has an implementation-defined signed or unsigned integer type as its underlying type. The values of type `wchar_t` can represent distinct codes for all members of ~~the largest extended~~ **any** character set specified among the supported locales (28.3.1).

Change in 6.8.2 [basic.fundamental] paragraph 11:

The types `char`, `wchar_t`, `char8_t`, `char16_t`, `char32_t` are collectively called *character types*. The character types, ~~Types~~ `bool`, ~~`char`, `wchar_t`, `char8_t`, `char16_t`, `char32_t`,~~ and the signed and unsigned integer types are collectively called *integral types*. A synonym for integral type is integer type. [Note: Enumerations (9.7.1) are not integral; however, unscoped enumerations can be promoted to integral types as specified in 7.3.6. — end note]

Change in 7.5.1 [expr.prim.literal] paragraph 1:

~~A *literal* is a primary expression.~~ The type of a *literal* is determined based on its form as specified in 5.13 [lex.literal]. A *string-literal* is an lvalue **designating the corresponding string literal object ([lex.string])**, a *user-defined-literal* has the same value category as the corresponding operator call expression described in 5.13.8 [lex.ext], and any other *literal* is a prvalue.

Change in 15.2 [cpp.cond] paragraph 12:

The resulting tokens comprise the controlling constant expression which is evaluated according to the rules of 7.7 using arithmetic that has at least the ranges specified in 17.3. For the purposes of this token conversion and evaluation all signed and unsigned integer types act as if they have the same representation as, respectively, intmax_t or uintmax_t (17.4). [Note: ... -- end note] This includes interpreting *character-literal*s, which may involve ~~converting escape sequences into execution character set members~~ **interpreting *escape-sequence*s and *universal-character-name*s (5.13.3 [lex.ccon])**. Whether the numeric value for these *character-literal*s matches the value obtained when an identical *character-literal* occurs in an expression (other than within a #if or #elif directive) is implementation-defined. [Note: ... -- end note] Also, whether a single-character *character-literal* may have a negative value is implementation-defined. Each subexpression with type `bool` is subjected to integral promotion before processing continues.

Change in 15.6.3 [lex.concat] paragraph 3:

For both object-like and function-like macro invocations, before the replacement list is reexamined for more macro names to replace, each instance of a ## preprocessing token in the replacement list (not from an argument) is deleted and the preceding preprocessing token is concatenated with the following preprocessing token. Placemarker preprocessing tokens are handled specially: concatenation of two placemarkers results in a single placemarker preprocessing token, and concatenation of a placemarker with a non-placemarker preprocessing token results in the non-placemarker preprocessing token. If the result is not a valid preprocessing token, the behavior is undefined. **If the result matches the syntax of a *universal-character-name*, the behavior is undefined.** The resulting token is available for further macro replacement. The order of evaluation of ## operators is unspecified.

Change in 16.3.3.3.5.1 [character.seq] paragraph 1:

The C standard library makes widespread use of characters and character sequences that follow a few uniform conventions:

- **Properties specified as *locale-specific* may change during program execution by a call to `setlocale(int, const char*)` (28.5.1 [clocale.syn]), or by a change to a `locale` object, as described in 28.3 [locales] and Clause 29 [input.output].**
- **The *execution character set* and the *execution wide-character set* are supersets of the basic literal character set (5.3 [lex.charset]). The encodings of the execution character sets and the sets of additional elements (if any) are locale-specific. [Note: The encoding of the execution character sets can be unrelated to any literal encoding. -- end note]**
- A *letter* is any of the 26 lowercase or 26 uppercase letters in the basic ~~execution~~ character set.
- The *decimal-point character* is the **locale-specific** (single-byte) character used by functions that convert between a (single-byte) character sequence and a value of one of the floating-point types. It is used in the character sequence to denote the beginning of a fractional part. It is represented in Clause 17 through Clause 32 and Annex D by a period, '.', which is also its value in the "C" locale~~, but may change during program execution by a call to `setlocale(int, const char*)`, [Footnote: ...] or by a change to a `locale` object, as described in 28.3 and Clause 29~~.

Change in 16.3.3.3.5.2 [multibyte.strings] paragraph 1:

A null-terminated multibyte string, or ntmbs, is an ntbs that constitutes a sequence of valid multibyte characters, beginning and ending in the initial shift state. [Footnote: An NTBS that contains characters only from the basic ~~execution~~ **literal** character set is also an NTMBS. Each multibyte character then consists of a single byte.]

Change in 27.13 [time.parse] table [tab:time.parse.spec]:

%Z	The time zone abbreviation or name. A single word is parsed. This word can only contain characters from the basic ~~source~~ character set (5.3 [lex.charset]) that are alphanumeric, or one of '_', '/', '-', or '+'.

Change in 28.4.2.2.3 [locale.ctype.virtuals] paragraphs 11 and 13:

The only characters for which unique transformations are required are those in the basic ~~source~~ character set (5.3 [lex.charset]).

[...]

For any character c in the basic ~~source~~ character set (5.3 [lex.charset]) the transformation is such that

```
do_widen(do_narrow(c, 0)) == c
```

Change in C.2.3 [diff.cpp14.lex]:

Affected subclause: 5.2
Change: Removal of trigraph support as a required feature.
Rationale: Prevents accidental uses of trigraphs in non-raw string literals and comments. Effect on original feature: Valid C ++ 2014 code that uses trigraphs may not be valid or may have different semantics in this revision of C ++ . Implementations may choose to translate trigraphs as specified in C ++ 2014 if they appear outside of a raw string literal, as part of the implementation-defined mapping from physical source file characters to the basic ~~source~~ character set.

Acknowledgements

Thanks to Corentin Jabot and his related paper P2297R0 for detailed discussions.